Тестовое задание для C++ разработчика [Middle]

Высокопроизводительный канал сжатых данных

1. Цель

Целью данного задания является разработка и реализация приложения на C++, демонстрирующего эффективную обработку данных и межпроцессное взаимодействие. Вы должны реализовать систему, которая считывает бинарный файл, сжимает его, передает через канал связи с ограниченным размером и декомпрессирует его на принимающей стороне.

Это задание предназначено для оценки ваших навыков в многопоточности и межпроцессном взаимодействии (IPC), сжатии данных, оптимизации производительности и проектировании систем на C++.

2. Сценарий

Представьте два отдельных **процесса**: **Производитель** и **Потребитель**.

- **Производитель** должен читать большой исходный файл, сжимать его содержимое и паралельно отправлять сжатые данные потребителю.
- **Потребитель** должен параллельно получать сжатые данные и декомпрессировать их для восстановления исходных данных.

Связь между этими двумя процессами ограничена **одним каналом связи**, реализованным через общую разделяемую память (shared memory).

3. Основные требования к алгоритму

Ваше приложение должно реализовать следующий конвейер:

1. **Вход:** Приложение должно принимать путь к исходному файлу в качестве аргумента командной строки. Данный файл представляет собой бинарное

представление чисел с плавающей точкой (32-бит) с разделителем `\n`. Во время тестов будет выбран файл размером до 50 МБ.

2. **Производитель: Многопоточное сжатие:**
   - Процесс должен читать исходный файл.
   - Процесс должен использовать несколько потоков для сжатия и передачи данных.
3. **Передача данных: Однопоточная общая память:**
   - Процесс должен использовать несколько потоков для приема и восстановления данных.
   - Сегмент общей разделяемой памяти строго ограничен **256 байтами**.

Вы можете свободно выбирать стратегию передачи данных в рамках разделяемой памяти и алгоритм сжатия.

4. Технические ограничения и условия

- **Язык:** C++ (14/17/20)
- **Платформа:** Linux (Debian-based OS)
- **Требования к сборке:** CMake; Clang; LLD
- **Зависимости:** Вы можете использовать стандартные библиотеки и известные библиотеки с открытым исходным кодом.
- **Целостность данных:** Допускается потеря данных (вы можете рассмотреть такие методы сжатия, для достижения цели по производительности).
- **Процессы:** Вы должны реализовать **Производителя** и **Потребителя** как отдельные процессы (два исполнительных файла).

5. Критерии приемки (критические)

Основным критерием успеха является **производительность**. Ваше решение будет считаться успешным только в том случае, если общее время, затраченное на весь конвейер (сжатие + передача + декомпрессия), будет **меньше**, чем время, которое потребовалось бы для передачи **несжатого** исходного файла через тот же 256-байтный канал общей памяти. Дополнительным критерием является процент потери данных после передачи.

Вы должны включить простой способ измерения и отображения:

1. Время сжатия и передачи данных от Производителя к Потребителю.
2. Время для передачи данных без сжатия.
3. Процент сжатия данных.
4. Процент потери данных.

6. Ожидаемые результаты

1. **Исходный код:** Ссылка на опубликованный проект (GitHub, GitLab, пр.) или tar.gz архив
2. **Инструкции:** Краткий файл README.md.
3. **Документация:** Короткий раздел в вашем README.md, объясняющий:
   - Выбор алгоритма сжатия; ход мыслей в процессе разработки.
   - Метод синхронизации общей памяти.
   - Компромиссы, на которые вы пошли для оптимизации.

Версия #12
Admin создал 22 сентября 2025 14:18:36
Admin обновил 24 февраля 2026 14:51:10